Automated Mobile Application Test Case Prioritization

Research Project Proposal • Most. Jannatul Ferdous • Computer Science & Engineering

Proposed by	Most. Jannatul Ferdous
Project type	Research / prototype study
Estimated duration	8–10 weeks
Max collaborators	3

Abstract

I want to investigate how machine-learning techniques can help prioritize test cases for mobile applications. The central idea is to identify which tests should be executed earlier based on information such as historical failures, recent changes, execution cost, and test coverage. The project will compare a practical baseline with learning-based prioritization and evaluate whether the approach can reduce testing effort while preserving useful fault-detection capability.

Research Motivation

Mobile applications can contain a large number of automated tests, while continuous testing environments often have limited time and computing resources. Running every test in an arbitrary order may delay the discovery of important failures. I want to explore whether historical and test-level information can be used to create a more useful execution order, especially for small and medium-sized mobile projects.

Research direction: The study will focus on prioritization rather than generating new test cases. A simple baseline will be established first, followed by one or more machine-learning approaches. The evaluation will emphasize early fault detection, execution cost, and the practical usefulness of the ranking.

Research Objectives

- Review established approaches for automated test case prioritization and mobile application testing.
- Prepare a dataset containing test-case characteristics, execution history, and failure information.
- Develop a baseline prioritization strategy for comparison.
- Develop and evaluate a machine-learning-based test prioritization approach.
- Measure whether important failing tests are executed earlier without excessive computational overhead.

Research Questions

RQ1. Which test-case and execution-history features are most useful for prioritization?

RQ2. Can machine learning improve early fault detection compared with a simple baseline?

RQ3. How does prioritization performance change when execution time is considered?

RQ4. How stable is the learned ranking when the application or test history changes?

Proposed Methodology

Step 1. Review relevant literature on regression testing, test case prioritization, mobile testing, and learning-based testing.

Step 2. Select a public or appropriately permitted mobile testing dataset/project containing test execution and failure information.

Step 3. Define test-level features such as historical failure rate, recent execution status, execution duration, code-change relevance, and coverage where available.

Step 4. Implement a baseline ordering strategy, such as historical-failure-first or execution-cost-aware ordering.

Step 5. Train a lightweight ranking or classification model to estimate the priority of each test case.

Step 6. Generate a ranked test suite and compare it with baseline orderings.

Step 7. Evaluate early fault detection, prioritization effectiveness, execution cost, and ranking stability.

Step 8. Perform error analysis to understand cases where the model places important tests too late.

Data and Experimental Resources

The project will use public or appropriately permitted mobile application test data. If a complete dataset is unavailable, a controlled experimental dataset can be constructed from an open-source mobile application and its test suite. The study will record only technical test information needed for evaluation and will not require personal user data.

Evaluation Plan

The primary evaluation will focus on how quickly failing tests are encountered under different test orders. Metrics such as Average Percentage of Faults Detected (APFD), time-aware fault detection, precision/recall for high-priority tests where applicable, and total execution cost may be used. Results will be compared across multiple test-ordering strategies rather than relying on a single model score.

Expected Challenges

- Historical test data may be incomplete or inconsistent.
- Failure patterns can change as the application evolves.
- A model may overfit to one application's testing history.
- Prioritizing short tests too aggressively could delay expensive but important tests.
- Different mobile projects may have different testing structures and feature definitions.

Reproducibility and Documentation

I will document the selected application or dataset, feature definitions, preprocessing decisions, baseline strategies, model configuration, evaluation metrics, and experimental settings. Test-ordering results will be saved so that the main comparisons can be repeated and independently reviewed.

Planned Mitigation

I will start with one clearly defined test suite and a simple baseline. If the available data is too limited for a reliable learning-based model, the project will be narrowed to a comparative prioritization study instead of making unsupported claims. Cross-project validation will be attempted only when the available data is sufficient.

Expected Outcomes

- A documented dataset and preprocessing pipeline for mobile test prioritization.
- A baseline test-ordering implementation.
- A lightweight machine-learning-based prioritization prototype.
- Quantitative comparison of prioritization effectiveness and execution cost.
- Failure-case analysis and recommendations for practical mobile regression testing.

Interest Tags

Machine Learning • Artificial Intelligence • Software Testing • Mobile Application Testing • Software Quality Assurance • Test Automation

Required Skills

Python • JavaScript • Data Analysis • Machine Learning • Software Testing • Test Automation • Mobile Development

Technical Stack

Python, pandas, scikit-learn, Jupyter Notebook, JavaScript, mobile testing frameworks where appropriate, and Git.

Potential Research Collaboration

I am looking for collaborators interested in software testing, machine learning, or mobile application development. A collaborator could contribute to dataset preparation, test automation, feature engineering, model evaluation, or research writing.

Scope and Responsible Research

The project will focus on authorized applications, public datasets, or open-source projects suitable for research. No unauthorized testing of third-party applications will be performed. The goal is to improve testing efficiency, not to make claims about the quality or security of individual developers or organizations.

Proposed Work Plan

Phase	Activities
1. Background study	Literature review, research question, dataset/project selection
2. Data preparation	Collect test history, define features, clean and structure data
3. Baseline	Implement and evaluate simple prioritization strategies
4. ML experiments	Train, compare, and validate lightweight models
5. Evaluation	APFD/time-aware metrics, cost analysis, error analysis
6. Reporting	Findings, limitations, reproducibility, documentation

What I Am Looking for in a Collaborator

I would like to work with someone who is interested in practical software testing and research experimentation. Contributions could include mobile test setup, dataset preparation, machine-learning experiments, metric implementation, or reviewing the research methodology.

Possible Next Step

The first step would be to select a suitable open-source mobile application or dataset, define the test-prioritization baseline, and agree on the evaluation protocol before beginning the machine-learning experiments.

Note: This document describes a proposed research project. It does not claim completed research, published results, or guaranteed improvement in testing efficiency. The scope and methodology can be refined through collaboration.